EduGenius v2.0 — Complete Help Manual

> ***Audience:*** *Anyone setting up, operating, or using EduGenius — from first clone to first student session.*
> ***Structure:*** *Part 1 deploys the system → Part 2 walks every agent and sub-agent → Part 3 walks connections → Part 4 walks the full student journey to course content.*
> ***Last updated:*** *2026-03-11*
> ***Live demo:*** *https://edugenius-ui.netlify.app*
> ***GitHub:*** *https://github.com/mathconcepts/edugenius-v2*

Table of Contents

1. System Overview
2. Prerequisites — What You Need Before You Start
3. Deployment — All Options From Scratch
   - 3.1 Local (Docker Compose)
   - 3.2 Hybrid (Supabase + Local)
   - 3.3 Railway PaaS
   - 3.4 GCP Cloud Run Recommended
   - 3.5 AWS ECS Fargate
   - 3.6 Netlify (Frontend Only)
4. First-Run Configuration
5. Connections — Mandatory and Optional
6. The 8 Agents — Deep Walk-Through
   - Scout · Atlas · Sage · Mentor · Herald · Forge · Oracle · Prism
7. Sub-Agents — Every Sub-Agent in Every Agent
8. The CEO Dashboard — All Pages and What They Do
9. Multi-User Setup — Students, Teachers, Parents, Admins
10. The Full Student Journey — Signup to Course Content
11. Batch Jobs and Automation
12. Skills System — VoltAgent Modules
13. Troubleshooting

---

1. System Overview

EduGenius v2.0 is an **AI-driven education SaaS** for competitive exam preparation in India (GATE EM, JEE, NEET, CAT, CBSE 12, UPSC). It operates as a network of 8 autonomous AI agents that generate content, tutor students, run marketing, monitor performance, and maintain infrastructure — all without manual intervention.

Architecture in one diagram

```
┌──────────────────────────────────────────────────────────────┐
│  CEO / Admin Dashboard  (React + Vite SPA)                   │
│  Student Interface · Teacher Panel · Parent View             │
└──────────────────────┬───────────────────────────────────────┘
                       │ HTTPS / REST / WebSocket
┌──────────────────────▼───────────────────────────────────────┐
│  Signal Bus (IndexedDB + optional Redis)                      │
│  8 agents communicate via typed signals                       │
│  ┌──────┐ ┌──────┐ ┌──────┐ ┌────────┐                      │
│  │Scout │ │Atlas │ │ Sage │ │ Mentor │                       │
│  └──────┘ └──────┘ └──────┘ └────────┘                      │
│  ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐                        │
│  │Herald│ │Forge │ │Oracle│ │Prism │                         │
```

```
│   └──────┘  └──────┘  └──────┘  └──────┘                          │
└──────────────────────────────────────────────────────────────────┘
                          │
                          │
┌─────────────────────────▼────────────────────────────────────────┐
│  LLM Layer (Gemini · Anthropic · OpenAI · Ollama)                 │
│  PostgreSQL (Supabase)   Redis   Supabase Vector/pgvector         │
└──────────────────────────────────────────────────────────────────┘
```

Key characteristics:

- All data persists in **PostgreSQL** (via Supabase or self-hosted)
- All inter-agent communication is via a **typed signal bus** — strongly typed, IndexedDB-backed, survives page reload
- The frontend is a standalone **React/Vite SPA** deployable to Netlify, Vercel, or any static host
- The backend is a **Node.js/Express API** deployable to Railway, GCP, AWS, or Docker
- AI calls go to whichever **LLM provider** has valid keys — falls back automatically

---

2. Prerequisites

Required software

Tool	Version	Purpose	Install
Node.js	≥ 20 LTS	Backend + frontend build	https://nodejs.org
npm	≥ 10	Package management	Comes with Node
Git	Any	Clone repository	https://git-scm.com
Docker	≥ 24	Local/hybrid deployments	https://docs.docker.com/desktop

> ***Shortcut:*** *Run* `./scripts/check-deps.sh --install` *after cloning — it audits and installs missing tools for you.*

Required API keys (minimum viable)

Key	Where to get it	Without it
`GEMINI_API_KEY`	https://aistudio.google.com	AI tutoring and content generation stop
`ANTHROPIC_API_KEY`	https://console.anthropic.com	Quality-critical tasks fall back to Gemini

> *You need **at least one** LLM provider key. The system cannot operate without any AI backend.*

Optional but strongly recommended

Key	Where to get it	What you lose without it
`SUPABASE_URL` + `SUPABASE_ANON_KEY`	https://supabase.com	Vector RAG and cloud DB (falls back to localStorage)
`VITE_WOLFRAM_APP_ID`	https://developer.wolframalpha.com	Mathematical fact-checking and Wolfram grounding
`NETLIFY_AUTH_TOKEN`	https://app.netlify.com	Automatic frontend deploy
Redis (`REDIS_URL`)	https://redis.io or Upstash	Caching (falls back to in-memory)

---

3. Deployment — All Options From Scratch

Step 0 — Clone and audit

```
git clone https://github.com/mathconcepts/edugenius-v2.git
cd edugenius-v2

# Run dependency check (reads your system, tells you what's missing)
./scripts/check-deps.sh

# Or: install everything missing in one shot
./scripts/check-deps.sh --install-all
```

3.1 — Local (Docker Compose) `deploy-local.sh`

Best for: Development, testing, demos, no internet required.
What runs: PostgreSQL 16 + Redis 7 + Backend API + Nginx frontend — all inside Docker on your machine.
Cost: $0

```
# First run — creates .env.local from the template
./scripts/deploy-local.sh
```

This creates `.env.local`. Edit it:

```
nano .env.local
```

Required fields to fill in:

```
GEMINI_API_KEY=your_key_here
ANTHROPIC_API_KEY=your_key_here    # optional but recommended
OPENAI_API_KEY=your_key_here       # optional
```

Then start the stack:

```
# Start everything
./scripts/deploy-local.sh

# With hot reload (recommended for development)
./scripts/deploy-local.sh --dev

# Reset the database and start fresh
./scripts/deploy-local.sh --reset

# Stop all services
./scripts/deploy-local.sh --down
```

Verify it's running:

```
curl http://localhost:3000/health
# Expected: {"status":"ok","timestamp":"..."}
```

Open frontend: http://localhost:80

Service ports:

Service	Port
Frontend	80
Backend API	3000
PostgreSQL	5432
Redis	6379

See also: `09-deployment.md` — full deployment reference

3.2 — Hybrid (Supabase + Local Backend) `deploy-hybrid.sh`

Best for: Current production setup. Local compute + cloud database + Netlify frontend.
What runs locally: Backend API + Redis.
What runs in cloud: PostgreSQL (Supabase) + Vector DB + Frontend (Netlify).
Cost: $0 (Supabase free tier + Netlify free tier)

Step 1 — Set up Supabase

1. Go to https://supabase.com and create an account
2. Click **New Project**
3. Note your project URL and API keys (Settings → API)

Step 2 — Configure

```
cp deploy/hybrid.env.example .env.hybrid
nano .env.hybrid
```

Fill in:

```
# Supabase
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_KEY=eyJ...    # for server-side operations

# LLM
GEMINI_API_KEY=your_key_here
ANTHROPIC_API_KEY=your_key_here

# Netlify (for frontend deploy)
NETLIFY_AUTH_TOKEN=nfp_...
NETLIFY_SITE_ID=your-site-id
```

Step 3 — Run Supabase migrations

```
# Apply all database schema migrations
./scripts/deploy-hybrid.sh --migrate

# OR manually using Supabase CLI
npx supabase db push --project-ref your-project-ref
```

Step 4 — Deploy

```
# Start local backend + connect to Supabase
./scripts/deploy-hybrid.sh

# With hot reload
./scripts/deploy-hybrid.sh --dev

# Stop
./scripts/deploy-hybrid.sh --down
```

Step 5 — Deploy frontend to Netlify

```
cd frontend
npm install
npm run build
npx netlify deploy --prod --dir=dist
```

Or connect GitHub repo in the Netlify dashboard for auto-deploy on every push.

Frontend environment variables (set in Netlify dashboard → Site settings → Environment variables):

```
VITE_API_URL=https://your-backend.railway.app    # or localhost:3000 for local
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=eyJ...
VITE_GEMINI_API_KEY=your_key_here
VITE_WOLFRAM_APP_ID=your_wolfram_id              # optional
```

3.3 — Railway PaaS `deploy-railway.sh`

Best for: Zero-ops. Managed Postgres + Redis + auto-scaling. Good for early production.
Cost: $20–60/month

Step 1 — Create Railway account

1. Go to https://railway.app
2. Sign up with GitHub (no credit card needed for trial)

Step 2 — Install Railway CLI (auto-installed by script)

```
npm install -g @railway/cli
railway login    # opens browser OAuth
```

Step 3 — Initialise project

```
# Creates Railway project, provisions Postgres + Redis, sets up services
./scripts/deploy-railway.sh --init
```

Step 4 — Add API keys

```
cp deploy/railway.env.example .env.railway
nano .env.railway    # add GEMINI_API_KEY, ANTHROPIC_API_KEY
./scripts/deploy-railway.sh --env-push
```

Step 5 — Deploy

```
./scripts/deploy-railway.sh
```

Railway builds the Dockerfile, provisions services, and gives you a public HTTPS URL automatically.

Auto-deploy on git push

```
# Link the repo
railway link
# Now every push to main triggers a redeploy
git push origin main
```

Useful commands

```
./scripts/deploy-railway.sh --logs      # stream live logs
./scripts/deploy-railway.sh --status    # deployment status
railway variables                        # list all env vars
railway open                             # open dashboard
```

3.4 — GCP Cloud Run □ Recommended `deploy-gcp.sh`

Best for: Production. Scales to zero overnight (saves money). Native Gemini AI integration. Best for Indian traffic (Mumbai region).
Cost: $15–40/month

Step 1 — Create GCP account

1. Go to https://console.cloud.google.com
2. Create account (new accounts get $300 free credit)
3. Create a new project at https://console.cloud.google.com/projectcreate
4. Note your **Project ID**

Step 2 — Full guided setup

```
./scripts/deploy-gcp.sh --setup
```

This interactively guides you through:

- Installing gcloud CLI
- `gcloud auth login` (browser OAuth)
- `gcloud auth application-default login`
- Enabling required APIs: Cloud Run, Artifact Registry, Secret Manager, Cloud Scheduler, Cloud SQL, GCS

Step 3 — Configure

```
cp deploy/gcp.env.example .env.gcp
nano .env.gcp
```

Variable	Required	Recommended value for India
`GCP_PROJECT_ID`	□	Your project ID
`GCP_REGION`	□	`asia-south1` (Mumbai)

`GEMINI_API_KEY`	□	Stored in Secret Manager
`CLOUD_RUN_MIN_INSTANCES`	No	`0` (scale to zero) or `1` (no cold start)

Step 4 — Deploy infrastructure, then app

```
# Provision: Artifact Registry + GCS + Secret Manager entries + Cloud Scheduler jobs
./scripts/deploy-gcp.sh --infra-only

# Full deploy: build Docker image → push → Cloud Run service
./scripts/deploy-gcp.sh
```

Step 5 — Cloud Scheduler (batch agents)

The deploy script automatically creates these scheduled jobs:

Job	Schedule (IST)	Agent
`atlas-content-gen`	Daily 7:30am	Atlas generates fresh content
`scout-market-scan`	Monday 11:30am	Scout scans market/competitors
`oracle-analytics`	Every 6 hours	Oracle computes KPIs
`herald-campaign`	Daily 1:30pm	Herald checks campaign performance
`mentor-engagement`	Daily 2:30pm	Mentor sends nudges
`forge-health`	Every 30 min	Forge checks system health

Useful commands

```
./scripts/deploy-gcp.sh --status    # deployment status
./scripts/deploy-gcp.sh --logs      # live log stream
gcloud run services list            # all Cloud Run services
gcloud scheduler jobs list          # all scheduler jobs
```

3.5 — AWS ECS Fargate `deploy-aws.sh`

Best for: Enterprise. Full control, VPC isolation, compliance requirements.
Cost: $50–80/month

Step 1 — AWS account and CLI

1. Go to https://aws.amazon.com and create an account
2. Install AWS CLI: https://docs.aws.amazon.com/cli/latest/userguide/install-cliv2.html
3. Configure credentials: `aws configure` (enter Access Key + Secret Key + region)

Recommended region for India: `ap-south-1` (Mumbai)

Step 2 — Deploy

```
cp deploy/aws.env.example .env.aws
nano .env.aws    # add AWS credentials + API keys
./scripts/deploy-aws.sh
```

This provisions:

- ECS Fargate cluster + task definitions
- Application Load Balancer
- RDS PostgreSQL (or connects to existing)
- ElastiCache Redis
- ECR container registry
- IAM roles with least-privilege policies

Step 3 — Configure SSL (required for production)

```
# Request a certificate in ACM (AWS Certificate Manager)
aws acm request-certificate --domain-name yourdomain.com --validation-method DNS

# After DNS validation, attach to ALB via console or
./scripts/deploy-aws.sh --ssl --domain yourdomain.com
```

3.6 — Netlify (Frontend Only)

Use this when your backend is already deployed elsewhere (Railway, GCP, AWS) and you only want to update the frontend.

```
cd frontend
npm install
npm run build

# First deploy (creates the site)
npx netlify deploy --prod --dir=dist

# Or connect the repo for auto-deploy
# 1. Go to app.netlify.com → Add new site → Import from Git
# 2. Build command: npm run build
# 3. Publish directory: dist
# 4. Add environment variables in Site settings
```

Required Netlify environment variables:

```
VITE_API_URL=https://your-backend-url.com
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=eyJ...
VITE_GEMINI_API_KEY=your_gemini_key
VITE_WOLFRAM_APP_ID=your_wolfram_id    # optional
```

---

4. First-Run Configuration

After any deployment, complete these steps before going live.

Step 1 — Access the CEO Dashboard

Navigate to your deployed URL (e.g., http://localhost, https://edugenius-ui.netlify.app, or your custom domain).

The default role is **CEO**. You'll see the full sidebar with all management features.

Step 2 — Create your first exam

Go to `/create-exam` (sidebar: Create Exam Wizard) and walk through:

1. **Exam selection:** Choose from GATE EM, JEE, NEET, CAT, CBSE 12, UPSC — or define a custom exam
2. **Subjects and topics:** Add/remove topics from the pre-populated catalogue
3. **Exam date:** Set the target exam date — this controls phase-aware content generation
4. **Content cadence:** Set daily questions, weekly blogs, videos per week, practice tests per month
5. **Difficulty distribution:** Easy/medium/hard percentages
6. **Language support:** Enable regional languages (9 supported)
7. **Review and approve:** Click **Approve** — this fires `EXAM_APPROVED` to all 8 agents simultaneously

> ***Exam approval is the trigger for everything.*** *Until you approve an exam, no agent does any work for that exam.*

See also: `14-exam-configuration.md` — full exam config structure

Step 3 — Configure integrations

Go to `/integrations` (CEO Integrations) and add connections. See Section 5 — Connections for the full list.

Step 4 — Add users

Go to `/user-portal` and add:

- Students (with their exam subscriptions)
- Teachers (assigned to subject areas)
- Parents (linked to student accounts)
- Admins (full access)

Step 5 — Set autonomy thresholds

Go to `/autonomy-settings` and configure:

- How much Atlas can spend per day (token budget)
- When Herald should pause campaigns (below CTR threshold)
- When Mentor should escalate to CEO (churn rate)
- Whether agents need CEO approval before publishing content

---

5. Connections — Mandatory and Optional

> ***See also:*** *`CEO-INTEGRATIONS-GUIDE.md` — full integration reference*

Mandatory connections (system cannot operate without these)

Connection	Variable	Purpose	How to get
LLM: Gemini	`GEMINI_API_KEY`	Powers all AI	https://aistudio.google.com
Database: PostgreSQL	`DATABASE_URL`	All persistent storage	Auto-provisioned by deploy scripts, or Supabase

Mandatory for payments (cannot charge students without)

Connection	Variable	Purpose	How to get
Razorpay (India)	`RAZORPAY_KEY_ID` + `RAZORPAY_KEY_SECRET`	INR payments	https://razorpay.com
OR Stripe (international)	`STRIPE_SECRET_KEY` + `STRIPE_PUBLISHABLE_KEY`	USD/card payments	https://stripe.com

Strongly recommended (major features degrade without these)

Connection	Variable	Purpose	Without it
Anthropic Claude	`ANTHROPIC_API_KEY`	Quality-critical tasks	Falls back to Gemini
Redis	`REDIS_URL`	Caching, rate limiting, pub/sub	Falls back to in-memory (single instance only)
Supabase Vector	`SUPABASE_URL` + `SUPABASE_ANON_KEY`	Vector RAG for PYQ search	Uses static bundle fallback
Wolfram Alpha	`VITE_WOLFRAM_APP_ID`	Mathematical fact verification	Math answers unverified
Netlify	`NETLIFY_AUTH_TOKEN` + `NETLIFY_SITE_ID`	Frontend deploy	Manual deploy only

Optional connections (enable specific features)

Connection	Variable	Purpose	Feature
OpenAI	`OPENAI_API_KEY`	GPT-4 fallback	Third LLM option
Ollama	`OLLAMA_URL`	Self-hosted models	Free, private LLM
Gmail / SMTP	`SMTP_HOST` + credentials	Transactional email	Password resets, reports
SendGrid	`SENDGRID_API_KEY`	Bulk email campaigns	Herald email campaigns
Twilio	`TWILIO_ACCOUNT_SID` + `TWILIO_AUTH_TOKEN`	WhatsApp/SMS	WhatsApp tutoring channel
Telegram Bot	`TELEGRAM_BOT_TOKEN`	Telegram channel	Telegram tutoring
Slack	`SLACK_WEBHOOK_URL`	Team alerts	CEO alerts in Slack
Google Analytics 4	`VITE_GA4_MEASUREMENT_ID`	Web analytics	Traffic tracking
Mixpanel	`MIXPANEL_TOKEN`	Product analytics	User event tracking
ElevenLabs	`ELEVENLABS_API_KEY`	Premium voice TTS	Sage voice output
Pinecone	`PINECONE_API_KEY`	Vector search	Alternative to Supabase vector
Manim Service	`VITE_MANIM_SERVICE_URL`	Math visualisations	Animated math in Sage

Connection priorities (fallback chain)

```
LLM:     Gemini Pro → Gemini Flash → Claude Sonnet → Claude Haiku → GPT-4 → GPT-3.5 → Ollama
Database: Supabase (cloud) → Local PostgreSQL → SQLite (dev only)
Cache:   Redis → In-memory (single node)
Vector:  Supabase pgvector → Pinecone → Static bundle (bundled PYQs)
Voice:   ElevenLabs → OpenAI TTS → Browser TTS (Web Speech API)
Email:   SendGrid → SMTP → Console log (dev)
```

---

6. Agents — Deep Walk-Through

> ***See also:*** *02-agent-architecture.md* *and* *18-agent-connection-map.md*

All 8 agents inherit from `BaseAgent`, which provides:

- **Heartbeat lifecycle** — periodic wake-up, signal processing, work execution
- **Budget management** — daily token limits with warning thresholds
- **State tracking** — `active / idle / busy / blocked / offline`
- **Signal bus integration** — subscribe to incoming signals, emit outgoing signals
- **Error recovery** — automatic retry with exponential backoff

---

6.1 — Scout (Market Intelligence)

Purpose: Monitors the external world. Feeds actionable intelligence to every other agent.
Heartbeat: Every 4 hours
Daily token budget: 30,000
Dashboard page: `/market-intel`

What Scout does:

- Scans Google Trends, Reddit (r/GATE, r/JEE, r/NEET, r/CATprep), education forums
- Tracks competitor pricing, features, and changes (Unacademy, BYJU's, Physics Wallah, etc.)
- Monitors exam syllabus changes, date announcements, and pattern shifts
- Finds SEO keyword opportunities (question-intent, comparison, PYQ-type queries)
- Analyses brand sentiment from YouTube comments and Twitter/X
- Monitors social platforms for unanswered student questions (Intent Scout)

Signals Scout receives:

- `PERFORMANCE_INSIGHT` from Oracle → What content is underperforming? Research why.
- `DEPLOY_METRICS` from Forge → Which topics just went live? Monitor SEO rankings.
- `CAMPAIGN_RESULT` from Herald → Which campaigns underperformed? Research audience.

Signals Scout emits:

- `TREND_SIGNAL` → Atlas: New keyword/PYQ pattern found; Atlas generates targeted content
- `KEYWORD_OPPORTUNITY` → Herald: High-volume keyword with low competition; run a campaign

---

6.2 — Atlas (Content Engine)

Purpose: Generates all educational content. The content factory of the platform.
Heartbeat: Every 30 minutes
Daily token budget: 200,000
Dashboard pages: `/atlas-workbench`, `/content-orchestrator`, `/content-hub`, `/content-strategy`

What Atlas does:

- Plans content by topic, gap, and phase (foundation / structured / sprint / post-exam)
- Writes lesson notes, summaries, worked examples, mnemonics
- Generates MCQs, numerical questions, assertion-reason, match-the-columns
- Creates formula cards, flash cards, revision sheets
- Optimises content for SEO (meta tags, headings, keyword density)
- Translates content into 9 regional languages
- Fact-checks all generated content before publication
- Generates content in multiple formats (worked_example, mcq, formula_card, flashcard, concept_map, analogy, short_video_script)

Content pipeline:

```
Request / Signal
    → Curator (topic selection, gap analysis)
    → Writer (draft lesson/question)
    → QuizMaster (MCQs and answers)
    → Visualizer (diagram specification)
    → SEOOptimizer (meta, headings, keywords)
    → FactChecker (accuracy verification)
    → Publish (emit CONTENT_READY → Sage)
```

Signals Atlas receives:

- `EXAM_APPROVED` from CEO → Start content generation for this exam
- `CONTENT_GAP` from Sage → Student can't understand a topic; generate alternative explanation
- `STRUGGLE_PATTERN` from Sage → Multiple students failing same concept; generate targeted content
- `FORMAT_REQUEST` from Lens/user → Channel/device needs a different format
- `FORMAT_SUCCESS` from Sage → This format worked; generate more like this
- `TREND_SIGNAL` from Scout → New keyword/PYQ found; generate content on it
- `ENGAGEMENT_GAP` from Mentor → Topic has low engagement; generate fresh variant
- `CONTENT_STALE` from Oracle → Engagement dropped; regenerate
- `FUNNEL_INSIGHT` from Prism → High-converting content type identified; prioritise it

Signals Atlas emits:

- `CONTENT_READY` → Sage: Content batch ready for accuracy verification
- `CONTENT_PUBLISHED` → Oracle: New content live; start performance tracking

Course Summary Outline (pre-generation approval):
Before any content is generated, CEO can go to `/course-orchestrator` → **Course Summary** tab and:

1. Generate a full hierarchical outline (Phase → Module → Topic → Lesson)
2. Toggle individual lessons on/off
3. Edit lesson titles, rationale, difficulty, and time estimates
4. Approve the outline → Atlas queues content generation in the approved order

See also: `21-course-summary-outline.md`

6.3 — Sage (AI Tutor)

Purpose: Tutors students in real time. The student-facing intelligence layer.
Heartbeat: Continuous (session-driven)
Daily token budget: 150,000
Student-facing page: `/chat`

What Sage does:

- Teaches via Socratic method — asks guiding questions rather than giving answers
- Adapts explanations to student archetype (Grinder / Strategist / Panicker / Casual / Topper)
- Adjusts difficulty in real time based on performance signals
- Detects frustration (consecutive wrong answers, session abandonment) and switches to empathy mode
- Enforces guard rails: blocks cheating requests, detects crisis signals, escalates to support
- Uses 6-phase structured thinking: Understand → Identify → Plan → Execute → Verify → Summarise
- Reads exam context (which exam, days to exam, weak topics) from the student persona
- Routes knowledge queries through the Knowledge Router before calling the LLM:
    - Wolfram Alpha (mathematical fact-checking)
    - Custom MCP endpoints
    - External subject APIs
    - Static PYQ bundles (GATE EM: 30 PYQs; CAT: 30 PYQs)
    - Supabase vector RAG (progressive indexer)

    - LLM fallback
- Logs every conversation to the Notebook (student can review later)
- Evaluates its own responses in real time via Live Evals (6 quality metrics)

Student archetypes Sage recognises:

- **The Grinder** — studies long hours, low efficiency; give structure and spaced repetition
- **The Strategist** — efficient, picks high-value topics; give analytics and gap analysis
- **The Panicker** — anxious, inconsistent; give reassurance and very small next steps
- **The Casual** — engages lightly; give engaging content and low-friction nudges
- **The Topper** — consistent, high-performing; give hard challenges and peer ranking

Signals Sage receives:

- `CONTENT_READY` from Atlas → New content verified and ready to use in sessions
- `STUDENT_STRUGGLING` from Mentor → Student stuck for N days; trigger doubt-clearing session
- `EXAM_APPROVED` from CEO → Ingest new exam context and PYQ bundles
- `STUDENT_ENROLLED` from UserService → Prepare personalised onboarding session

Signals Sage emits:

- `CONTENT_GAP` → Atlas: Student can't understand something; generate alternative
- `STRUGGLE_PATTERN` → Atlas: Multiple students failing same concept
- `FORMAT_SUCCESS` → Atlas: This format worked; generate more like this
- `MASTERY_ACHIEVED` → Oracle + Mentor: Student mastered a topic
- `BREAKTHROUGH` → Oracle + Mentor: Significant insight moment
- `FRUSTRATION_ALERT` → Mentor: Student showing frustration
- `BEHAVIORAL_SNAPSHOT` → Oracle: Session-level data for analytics
- `CONTENT_VERIFIED` → Forge + Herald: Sage has confirmed content accuracy

Skills wired into Sage:

- **Guard Rails** — content safety, cheating detection, crisis detection
- **Thinking Tool** — 6-phase structured reasoning for complex problems
- **Live Evals** — real-time quality scoring (accuracy, clarity, pedagogical, brevity, engagement, curriculum fit)
- **Voice Skill** — TTS via ElevenLabs or browser speech synthesis
- **User Research** — student archetype profile injected into system prompt every session

---

6.4 — Mentor □□ (Engagement)

Purpose: Keeps students coming back. Nudges, re-engages, celebrates, and reports to parents.
Heartbeat: Every 2 hours
Daily token budget: 50,000
Dashboard page: `/students`

What Mentor does:

- Predicts churn risk using Oracle signals + behavioral patterns
- Sends personalised nudges via WhatsApp, Telegram, or in-app notifications
- Tracks study streaks and triggers streak-recovery nudges when broken
- Manages milestone celebrations (first session, first quiz pass, first mastery, 30-day streak)
- Re-engages dormant students with personalised messages referencing their last topic
- Sends weekly progress reports to parents
- Routes student-struggling cases back to Sage for targeted doubt-clearing
- Adapts nudge tone by student archetype:
    - Panicker → calm, reassuring, "You've handled harder before"
    - Grinder → technique tips, "Let's make your time count"
    - Casual → social proof, "3 students in your batch scored 80+ this week"

Signals Mentor receives:

- `MASTERY_ACHIEVED` from Sage → Celebrate with the student immediately
- `FRUSTRATION_ALERT` from Sage → Send empathy message before next session
- `BREAKTHROUGH` from Sage → Celebrate and push to next level
- `CHURN_RISK` from Oracle → Trigger re-engagement within 2 hours
- `CHURN_COHORT_ALERT` from Oracle → Batch re-engagement for at-risk cohort
- `SR_OVERDUE` from Lens → Spaced repetition review overdue; send reminder
- `EXAM_DEPLOYED` from Forge → Begin student onboarding for new exam
- `EXAM_APPROVED` from CEO → Configure nudge rules for this exam
- `STUDENT_ENROLLED` from UserService → Start onboarding sequence
- `STUDENT_STRUGGLING` from Mentor (self-escalation) → Route to Sage
- `FUNNEL_INSIGHT` from Prism → Activation leak; adjust onboarding nudge

Signals Mentor emits:

- `STUDENT_STRUGGLING` → Sage: Student stuck for N days; trigger doubt-clearing
- `ENGAGEMENT_GAP` → Atlas: Topic has persistent low engagement; generate fresh content

---

6.5 — Herald (Marketing)

Purpose: Launches and optimises all marketing campaigns. Responds to performance data in real time.
Heartbeat: Every 2 hours
Daily token budget: 40,000
Dashboard page: `/content-hub`, `/social-intent`

What Herald does:

- Creates and publishes SEO-optimised blog posts for each exam
- Runs WhatsApp, LinkedIn, Instagram, YouTube Shorts, and Google Ads campaigns
- Generates platform-specific ad copy with A/B variants
- Monitors campaign CTR/ROAS and kills underperformers automatically
- Answers student questions on Reddit, Quora, YouTube comments (Social Intent Scout pipeline)
- Manages referral and affiliate programs
- Creates weekly email newsletters
- Pitches PR stories to education media

Social Intent Scout pipeline (5-stage automated system):

```
IntentScout (monitors Reddit/Quora/X/YouTube for student questions)
    → AnswerCrafter (generates humanised, SEO-grade answers)
    → HookSmith (adds platform-specific CTAs)
    → ApprovalGate (routes through admin review or auto-approves)
    → PostScheduler (posts at optimal IST times)
```

Signals Herald receives:

- `CONTENT_VERIFIED` from Sage → Content approved; promote it
- `EXAM_DEPLOYED` from Forge → Exam live; launch campaigns
- `CAMPAIGN_PERFORMANCE` from Oracle → CTR/ROAS data; adjust or kill campaigns
- `KEYWORD_OPPORTUNITY` from Scout → High-volume keyword; create targeted campaign
- `EXAM_APPROVED` from CEO → Prepare campaign calendar
- `FUNNEL_INSIGHT` from Prism → Acquisition funnel leak; rewrite CTAs

Signals Herald emits:

- `MARKETING_LIVE` → Oracle: Campaigns launched; start tracking performance
- `CAMPAIGN_RESULT` → Scout: Campaign underperformed; research why

6.6 — Forge ⚙ (DevOps)

Purpose: Deploys, maintains, and monitors all infrastructure. CI/CD, CDN, and health checks.
Heartbeat: Every 15 minutes
Daily token budget: 20,000
Dashboard page: `/status`

What Forge does:

- Deploys new content and exam configurations to the CDN
- Runs database migrations when schema changes are needed
- Manages build pipeline (test → build → push → deploy)
- Keeps CDN cache warm for high-traffic topics
- Monitors system health: API response times, DB connections, Redis memory, error rates
- Guards rollback: if a deploy causes errors, automatically rolls back to last good state
- Monitors SSL certificate expiry and triggers renewal
- Reports build/deploy metrics to Scout for SEO monitoring

Signals Forge receives:

- `CONTENT_VERIFIED` from Sage → Deploy verified content to CDN
- `EXAM_APPROVED` from CEO → Provision infrastructure for new exam

Signals Forge emits:

- `EXAM_DEPLOYED` → Oracle + Herald + Mentor: Infrastructure live; begin tracking, campaigns, onboarding
- `DEPLOY_METRICS` → Scout: Deployment complete; monitor SEO rankings for new pages

---

6.7 — Oracle (Analytics)

Purpose: Tracks everything. Drives all feedback loops. The intelligence hub of the platform.
Heartbeat: Every 15 minutes
Daily token budget: 60,000
Dashboard pages: `/analytics`, `/exam-analytics`, `/prism`, `/revenue`

What Oracle does:

- Tracks every student interaction: session durations, question attempts, correct/incorrect, skips
- Computes real-time mastery scores using Bayesian Knowledge Tracing (BKT)
- Calculates churn risk scores for every student
- Tracks cohort patterns: which topics have low engagement across many students
- Monitors marketing funnel: impressions → clicks → signups → paid → retained
- Tracks content performance: which atoms have highest engagement/mastery correlation
- Generates weekly KPI reports for the CEO briefing
- Computes A/B test results (content variant experiments)
- Exports journey events to Prism for funnel analysis

Signals Oracle receives:

- `MASTERY_ACHIEVED` from Sage → Record, update student progress
- `BREAKTHROUGH` from Sage → Record significant event
- `BEHAVIORAL_SNAPSHOT` from Sage/Lens → Session-level data ingestion
- `EXAM_DEPLOYED` from Forge → Set up analytics funnels for new exam
- `MARKETING_LIVE` from Herald → Begin campaign performance tracking
- `STUDENT_ENROLLED` from UserService → Set up student analytics profile
- `CONTENT_PUBLISHED` from Atlas → Set up content performance tracking

Signals Oracle emits:

- `CHURN_RISK` → Mentor: Individual student at risk of dropping off

- `CHURN_COHORT_ALERT` → Mentor: Cohort-level churn risk
- `CONTENT_STALE` → Atlas: Topic engagement dropped; regenerate
- `PERFORMANCE_INSIGHT` → Scout: Weekly data; investigate gaps
- `CAMPAIGN_PERFORMANCE` → Herald: CTR/ROAS data for campaign optimisation
- Journey event export → Prism: Raw funnel data for journey analysis

---

6.8 — Prism (Journey Intelligence)

Purpose: Maps the complete user journey. Detects funnel leaks. Routes insights to the agent responsible for each leak.
Heartbeat: Every hour (triggered by Oracle export)
Daily token budget: 25,000
Dashboard page: `/prism`

What Prism does:

- Receives raw journey event exports from Oracle (entry points, drop-off stages, conversion events)
- Builds full user journey traces: acquisition channel → first session → first question → first purchase → 30-day retention
- Detects funnel leaks by stage and user segment:
  - Acquisition: which channels have poor conversion from click → signup?
  - Activation: where do new students drop off in onboarding?
  - Content: which content types keep students longest vs. bounce fastest?
- Emits `FUNNEL_INSIGHT` signals to the specific agent responsible for each leak

Prism workflow — `prism_analysis` (4-step pipeline):

```
Step 1: Oracle exports journey data to Prism
Step 2: Prism analyses traces, detects leaks by stage
Step 3: Prism emits FUNNEL_INSIGHT to responsible agent:
           - Acquisition leak → Herald (fix CTAs, landing pages)
           - Activation leak → Mentor (fix onboarding nudge)
           - Content signal → Atlas (prioritise high-converting format)
Step 4: Herald/Mentor/Atlas act on the insight
```

Signal Prism emits:

- `FUNNEL_INSIGHT` → Herald (acquisition leak): rewrite CTAs or landing page copy
- `FUNNEL_INSIGHT` → Mentor (activation leak): adjust onboarding nudge sequence
- `FUNNEL_INSIGHT` → Atlas (content signal): prioritise the format that converts best

---

7. Sub-Agents — Complete Registry

> ***See also:*** *`02-agent-architecture.md` — full 45+ sub-agent YAML specs*

Scout Sub-Agents (5)

Sub-Agent	What it does	Trigger
TrendSpotter	Monitors Google Trends, Reddit, forum hot topics for emerging exam keywords	Every 4 hours
CompetitorTracker	Scrapes competitor pricing pages, feature lists, and review sites	Daily
ExamMonitor	Watches official exam board sites for syllabus/date changes	Daily
KeywordHunter	Finds high-volume, low-competition SEO keywords in education niche	Weekly

SentimentScanner	Analyses YouTube comments and Twitter/X for brand sentiment and product feedback	On request

Atlas Sub-Agents (7)

Sub-Agent	What it does	Trigger
Curator	Selects topics for content generation based on gaps, demand signals, and exam phase	On request
Writer	Generates lesson notes, summaries, analogies, worked examples	On request
QuizMaster	Generates MCQs, numerical, assertion-reason, match-the-columns, PYQ-style	On request
Visualizer	Creates Manim-compatible diagram specifications and infographic plans	On request
SEOOptimizer	Writes meta titles, descriptions, headings, and keyword placements	Post-draft
Translator	Converts content to Hindi, Tamil, Telugu, Kannada, Bengali, Marathi, Malayalam, Gujarati, Punjabi	On request
FactChecker	Verifies mathematical accuracy, formula correctness, and factual claims	Before publish

Sage Sub-Agents (7)

Sub-Agent	What it does	Trigger
Socratic	Asks probing questions to guide the student to the answer themselves	All sessions
Explainer	Delivers clear, structured explanations adapted to student level	When student asks "explain"
ProblemSolver	Works through numerical/derivation problems step by step	Problem-solving sessions
ConceptMapper	Draws connections between topics and builds mental models	After topic mastery
PracticeCoach	Guides timed MCQ practice with post-session analysis	Practice sessions
EmotionReader	Detects frustration, anxiety, and confidence from message patterns	All sessions (background)
LanguageAdapter	Switches explanation language and complexity based on student preference	On request or auto

Mentor Sub-Agents (6)

Sub-Agent	What it does	Trigger
ChurnPredictor	Scores each student's churn risk using session patterns	Every 2 hours
NudgeEngine	Writes personalised nudge messages adapted to archetype and channel	When nudge is triggered
StreakTracker	Monitors daily study streaks and triggers recovery nudges on day 1 of absence	Daily

MilestoneManager	Tracks and celebrates achievements (first topic, first 80+ score, streaks)	Event-driven
ReEngager	Sends personalised re-engagement sequences for dormant students (T+0, T+1, T+3, T+7)	CHURN_RISK signal
ParentReporter	Generates weekly PDF-style progress reports for parents	Weekly

Herald Sub-Agents (7)

Sub-Agent	What it does	Trigger
CampaignManager	Plans, launches, and monitors ad campaigns across platforms	Post EXAM_DEPLOYED
SocialPoster	Publishes content to Instagram, LinkedIn, YouTube, Twitter/X	Daily
EmailCrafter	Writes weekly newsletters and drip email sequences	Weekly
LeadNurturer	Manages lead scoring and personalised follow-up sequences	On lead capture
ReferralManager	Runs referral/affiliate program — tracking, payouts, optimisation	Ongoing
PRCoordinator	Pitches stories to EdTech media and education journalists	Monthly
InfluencerFinder	Discovers relevant YouTubers and educators for partnerships	Scout signal

Forge Sub-Agents (7)

Sub-Agent	What it does	Trigger
BuildRunner	Runs test suite + production build when new content is verified	CONTENT_VERIFIED
TestOrchestrator	Runs unit, integration, and E2E tests before deploy	Pre-deploy
CDNSyncer	Pushes built assets to CDN and warms caches for top topics	Post-build
CacheManager	Manages Redis TTLs, clears stale content, optimises hit rates	Every 30 min
DBMigrator	Runs Supabase/PostgreSQL migrations safely with rollback protection	Schema change
RollbackGuard	Monitors error rate after deploy; rolls back automatically if errors spike	Post-deploy
HealthChecker	Checks API uptime, DB connections, Redis memory, SSL expiry	Every 30 min

Oracle Sub-Agents (6)

Sub-Agent	What it does	Trigger
MetricTracker	Records all student events to the analytics store	Real-time
AnomalyDetector	Flags unusual patterns: sudden drop in sessions, error rate spike	Continuous
ReportGenerator	Produces CEO weekly briefing reports with KPIs and trend analysis	Weekly

FunnelAnalyzer	Tracks acquisition → activation → retention funnel metrics	Daily
CohortAnalyzer	Groups students by archetype/exam/join date and finds cohort patterns	Weekly
ABEvaluator	Computes statistical significance of A/B content experiments	When experiment has enough data

---

8. The CEO Dashboard — All Pages and What They Do

The dashboard is role-aware. The sidebar changes based on role (CEO / Teacher / Student / Parent / Admin / Manager).

CEO Role — Full page list

URL	Page name	Purpose
`/`	Dashboard	Overview: active students, daily sessions, content stats, agent status
`/briefing`	CEO Briefing	Daily AI-generated brief: what happened, what needs attention, recommended actions
`/create-exam`	Exam Creation Wizard	Multi-step wizard to configure and launch a new exam
`/course-orchestrator`	Course Orchestrator	Plan content phases; Course Summary tab for pre-approval outline
`/atlas-workbench`	Atlas Workbench	Directly trigger Atlas content generation for any topic
`/content-hub`	Content Hub	Multi-format content generation, repurposing, and campaign creation
`/content-orchestrator`	Content Orchestrator	Manage running content generation pipelines
`/content-strategy`	Content Strategy	Long-term content calendar and topic gap analysis
`/content-intelligence`	Content Intelligence	Which content atoms are performing best/worst
`/market-intel`	Market Intelligence	Scout's live market scan results: trends, competitors, keywords
`/opportunity-discovery`	Opportunity Discovery	AI-surfaced growth opportunities (user and market)
`/social-intent`	Social Intent Scout	Monitor and respond to student questions on Reddit/Quora/X/YouTube
`/page-builder`	Page Builder	Build and deploy custom HTML landing pages to Netlify
`/analytics`	Analytics	Student performance charts, session data, mastery heatmap
`/exam-analytics`	Exam Analytics	Per-exam breakdown: enrollment, completion, score distribution
`/prism`	Prism Dashboard	User journey funnel: where students drop off
`/revenue`	Revenue Dashboard	Subscription revenue, churn rate, LTV, MRR/ARR
`/growth-command`	Growth Command	Unified growth metrics: acquisition, activation, retention, revenue

`/students`	Students	Full student roster with engagement scores and nudge history
`/user-portal`	User Management Portal	Add/edit students, teachers, parents; manage subscriptions
`/integrations`	Integrations	Add/manage all external connections (LLM, payment, email, chat)
`/connections`	Connection Registry	MCP and external API knowledge source management
`/user-attributes`	User Attributes	Custom user attributes and segmentation
`/autonomy-settings`	Autonomy Settings	Set thresholds for how much each agent can do autonomously
`/strategy`	Strategy	CEO strategy board: OKRs, experiments, roadmap
`/agent-skills`	Agent Skills	Toggle VoltAgent skill modules; manage prompt templates; User Research Report
`/agents`	Agents	Live agent status: which agents are active, heartbeat logs, signal history
`/trace`	Trace Viewer	Step-by-step execution trace for any agent run
`/status`	System Status	Health dashboard: API, DB, Redis, CDN, all agent uptime
`/batch-generate`	Batch Generate	Trigger Atlas batch content generation by exam and topic set
`/settings`	Settings	Platform-wide settings: branding, notifications, billing
`/blog`	Blog Admin	Preview and publish SEO blog posts created by Herald
`/events`	Events	Signal bus event log: all inter-agent signals in real time

Teacher Role — Pages visible

URL	Purpose
`/`	Teacher dashboard: class progress overview
`/students`	Their assigned students
`/content`	View/rate content for their subject
`/analytics`	Analytics for their class
`/chat`	Sage tutor (their own access)
`/notebook`	Notes and saved explanations

Student Role — Pages visible

URL	Purpose
`/`	Student dashboard: today's plan, exam countdown, streak
`/chat`	Sage AI tutor — main learning interface
`/learn`	Browse topics, lessons, formula cards
`/practice`	Timed MCQ practice sessions
`/progress`	Personal progress: mastery by topic, score trends

`/notebook`	AI-logged conversation summaries, bookmarks
`/insights`	Personalised insights: strengths, gaps, predicted score
`/feedback`	Rate Sage responses and flag errors

Parent Role — Pages visible

URL	Purpose
`/`	Child progress summary
`/progress`	Child's study hours, topics covered, score trends
`/insights`	AI-generated parent report

---

9. Multi-User Setup — Students, Teachers, Parents, Admins

User types and how to create them

All users are managed through `/user-portal` (CEO/Admin access).

Student accounts:

A student account requires:

- Name, email or phone number (for OTP login)
- Auth method: email OTP, WhatsApp OTP, Telegram, passkey, or Google SSO
- Exam subscription(s): link to one or more active exams
- Channel preference: in-app, WhatsApp, or Telegram

```
In /user-portal:
1. Click "Add User"
2. Set role = Student
3. Add Exam Subscription → select exam → select plan (Free/Pro/Premium)
4. Set channel preference
5. Save → student receives OTP welcome message on their preferred channel
```

Multiple exam subscriptions (Student): A student can subscribe to multiple exams (e.g., JEE + GATE EM). When they have multiple active subscriptions:

- A **pill switcher** appears in the Sage chat interface to switch active exam context
- Sage adapts its knowledge base, PYQ context, and prompts to the selected exam
- Each exam has independent progress tracking, spaced repetition queue, and analytics

Teachers:

```
role = Teacher
→ Assign subject areas (e.g., Physics for JEE)
→ Sage uses teacher's assigned subjects when flagging content gaps
→ Teacher gets class-level analytics
```

Parents:

```
role = Parent
→ Link to child's student account via childStudentId
→ Parent receives weekly report via email/WhatsApp
→ Parent can view child's dashboard in read-only mode
```

Admins:

```
role = Admin
→ Full CEO-equivalent access
→ Can approve social intent answers in the ApprovalGate
→ Can manage all users
```

Managers:

```
role = Manager
→ Access to /manager — team-level view
→ Can see teacher performance and cohort analytics
```

Login flow

Students/teachers/parents log in at `/login`. Supported methods:

- **Email OTP** — enter email, receive 6-digit OTP (demo: use `000000`)
- **WhatsApp OTP** — enter phone number, receive OTP on WhatsApp via Twilio
- **Telegram** — start the bot, authenticate
- **Passkey** — WebAuthn (biometric/device PIN)
- **Google SSO** — one-click Google login

First login for students:

- If no active exam subscription → `ExamSelectModal` appears to choose exam
- Student selects exam and plan → redirected to Student Dashboard

Channel-specific UX

Channel	What works	What's different
In-app	Full feature set	Richest experience, all pages
WhatsApp	Chat with Sage, receive nudges, get progress updates	Text-only, concise answers, emoji-friendly
Telegram bot	Same as WhatsApp + `/exam [name]` command to switch exam	Async messaging, quick commands

---

10. The Full Student Journey — Signup to Course Content

This section walks the complete path from a new user hearing about EduGenius to receiving personalised course content.

Stage 1 — Awareness (Herald + Scout)

What happens:

- Scout identifies a student question on Reddit or Quora: "*Best way to study Electromagnetism for GATE?*"
- IntentScout classifies it as `exam_prep_question` intent
- AnswerCrafter generates a helpful, humanised answer with an EduGenius CTA
- ApprovalGate reviews (CEO-configured: auto-approve if confidence > 0.85)
- PostScheduler posts the answer at optimal IST time (e.g., 9pm–11pm for GATE students)
- Herald simultaneously runs a Google Ads campaign on the keyword "GATE EM study plan"
- Student sees the answer or ad and clicks through to the EduGenius landing page

What the student sees:

- `/website` — marketing home page with exam selector
- `/website/exams/gate-em` — GATE EM exam-specific page with features, testimonials, pricing
- `/website/pricing` — pricing page with Free/Pro/Premium tiers

Stage 2 — Signup and Login

URL: `/login` or `/website/signup`

1. Student clicks **Start Free**
2. Chooses auth method (email OTP, WhatsApp OTP, Google SSO, etc.)
3. Enters credentials → OTP delivered to preferred channel
4. Enters OTP (demo code: `000000`)
5. **First-login gate:** ExamSelectModal appears
   - Student selects exam (e.g., GATE EM)
   - Student selects plan (Free: 5 questions/day; Pro: unlimited; Premium: live sessions)
6. Exam subscription created → `STUDENT_ENROLLED` signal emitted
7. Signal received by: **Mentor** (start onboarding), **Sage** (prepare context), **Oracle** (create analytics profile)

Stage 3 — Onboarding (Mentor)

What Mentor does immediately:

1. Sends welcome message on preferred channel:
   - WhatsApp: "*Welcome to EduGenius! Your GATE EM journey starts now. Your first session is ready: Linear Algebra basics. Tap to begin *"
   - In-app notification: similar message with direct link
2. Sends 5-message onboarding sequence over the next 3 days:
   - T+0: Welcome + first topic prompt
   - T+1: "Did you try your first session? Here's a 5-min formula card for Eigen Values"
   - T+3: "3 students who studied 30 min on Day 1 are 2× more likely to reach 70+ on GATE EM"
   - T+7 (if no activity): Re-engagement — "Where did you go? Your Laplace Transform explanation is still here "

What Oracle does immediately:

1. Creates student analytics profile
2. Sets up funnel tracking for this student's journey
3. Begins churn risk monitoring (baseline: 30-day dropout rate)

Stage 4 — First Session with Sage

URL: `/chat`

Student sees the Sage chat interface. Before responding, Sage does the following (invisible to student):

Step 1 — Load student context:

```
loadCurrentUser() → exam = GATE_EM, plan = FREE, daysToExam = 45
loadPersona() → archetype = the_casual, weak = [Linear Algebra], strong = [Calculus]
resolveActiveExam() → GATE_EM (from sessionStorage or first active subscription)
```

Step 2 — Knowledge routing (for every query):

```
resolveKnowledge(query, allowedSources):
  1. Wolfram Alpha → mathematical queries (if VITE_WOLFRAM_APP_ID set)
  2. Custom MCP endpoints → if configured
  3. External exam APIs → if configured
```

```
  4. Static PYQ bundles → GATE EM: 30 PYQs from gateEmPyqContext.ts
  5. Supabase vector RAG → if SUPABASE_URL set (progressive indexer)
  6. LLM fallback → Gemini/Claude generates from training data
```

Step 3 — Guard Rails check:

```
guardRailsSkill.check(message):
  - Cheating? (asking for full exam paper, answer keys) → blocked
  - Crisis? (self-harm language) → escalated to support
  - Off-topic? → redirected to study
  - Pass → proceed
```

Step 4 — Thinking Tool (for complex questions):

```
ThinkingTool.resolve(query):
  - identify_concept? → structure 6-phase reasoning
  - solve_math? → inject THINKING_DIRECTIVE into Sage prompt
```

Step 5 — Build system prompt:

```
buildSageSystemPrompt(persona, knowledgeContext, exam, daysToExam):
  - Socratic framing
  - Exam phase (foundation/structured/sprint/post-exam)
  - Student archetype instructions
  - Knowledge context from router
  - PYQ examples from static bundle
  - Active topic: Linear Algebra
```

Step 6 — LLM call:

```
llmService.generate(systemPrompt, userMessage) → Gemini Flash or Claude Haiku
```

Step 7 — Live Evals (after each response):

```
liveEvalsSkill.score(response):
  - Accuracy: Does it match verified knowledge?
  - Clarity: Is it understandable?
  - Pedagogical: Does it teach, not just tell?
  - Brevity: Is it appropriately concise?
  - Engagement: Is the tone motivating?
  - Curriculum fit: Is it relevant to the exam?
→ score logged to Oracle
```

Step 8 — Log to Notebook:

```
Sage session summary automatically added to /notebook
Student can review all explanations, bookmark important ones
```

What the student sees:

The Sage chat renders as:

- User messages: gradient pill bubbles (right-aligned)
- Sage messages: clean text with agent avatar + name badge (left-aligned)
- Math equations: rendered via KaTeX
- Code: syntax-highlighted via Prism.js
- Diagrams: Manim visualisations if service is running

- Voice: TTS playback button on each response (if voice skill enabled)

Stage 5 — Topic Learning Flow (`/learn`)

URL: `/learn` or `/learn/linear-algebra`

Student can browse topics for their active exam:

1. **Topic list** — organised by exam phase (Foundation, Structured, Sprint)
2. **Topic page** — shows:
   - Lesson notes (generated by Atlas Writer)
   - Formula card (generated by Atlas)
   - 5–10 MCQs (generated by Atlas QuizMaster)
   - Worked examples (2–3 per topic)
   - PYQ section (previous year questions from static bundle)
   - Manim visualisation (if Manim service is running)
3. **Mark as read** → Sage updates mastery estimate
4. **Start practice** → redirects to `/practice` pre-filtered to this topic

---

Stage 6 — Practice Sessions (`/practice`)

URL: `/practice`

1. Student selects exam and topic(s)
2. Selects session type: timed (20 min) or untimed
3. Selects difficulty: easy / mixed / hard
4. MCQ questions shown one at a time
5. On each answer:
   - Correct: Sage (via Practice Coach) shows brief reinforcement
   - Incorrect: Sage applies Socratic method — "What's the formula for this type?"
   - Hint requested: Sage gives a guiding question, not the answer
6. End of session:
   - Score shown
   - Topic mastery updated (BKT algorithm)
   - Weak questions flagged for spaced repetition
   - `MASTERY_ACHIEVED` or `FRUSTRATION_ALERT` emitted as appropriate

Stage 7 — Progress and Insights (`/progress` , `/insights`)

URL: `/progress`

Student sees:

- Mastery heatmap: all exam topics coloured by mastery level (green/yellow/red)
- Score trend: practice test scores over time
- Study hours: daily/weekly chart
- Streak counter: consecutive study days
- Predicted score: AI-estimated GATE score based on current mastery trajectory
- Next recommended topic: highest-impact gap topic

URL: `/insights`

Oracle-generated insights:

- "You've mastered 6/10 Linear Algebra topics. At your current pace, you'll complete it in 4 days."
- "Your weakest topic is Complex Variables. 3 students who focused here for 1 week improved 15 marks."
- "You study best between 9pm–11pm. Your morning sessions have 40% lower accuracy."

Stage 8 — Course Content — How It's Generated and Delivered

This is the complete content pipeline, end-to-end:

How content is generated for a new exam:

```
CEO approves exam in /create-exam wizard
    │
    ├── EXAM_APPROVED → Scout
    │   → begins monitoring keywords for this exam
    │
    ├── EXAM_APPROVED → Atlas
    │   → Curator plans content: Foundation phase first
    │     (Linear Algebra → Calculus → Differential Equations → ...)
    │   → Writer drafts lesson notes for each topic
    │   → QuizMaster generates 15 MCQs per topic
    │   → FactChecker verifies mathematical accuracy
    │   → CONTENT_READY → Sage
    │
    ├── EXAM_APPROVED → Sage
    │   → ingests exam PYQ bundle
    │   → sets exam context for all student sessions
    │
    ├── EXAM_APPROVED → Forge
    │   → provisions CDN paths for new exam
    │   → sets up deployment pipeline
    │
    ├── EXAM_APPROVED → Herald
    │   → prepares blog calendar for exam
    │   → queues first SEO articles
    │
    ├── EXAM_APPROVED → Oracle
    │   → creates analytics funnels for this exam
    │   → sets up BKT tracking per topic
    │
    └── EXAM_APPROVED → Mentor
        → configures nudge rules for exam timeline
```

Content atoms generated per topic:

Atom type	Generator	What it is
`lecture_notes`	Atlas Writer	500–1500 word topic overview with examples
`formula_sheet`	Atlas Writer	Condensed formulas, no derivations
`worked_example`	Atlas Writer	2–3 step-by-step solved problems
`mcq`	Atlas QuizMaster	15 MCQs per topic, 3 difficulty levels
`numerical`	Atlas QuizMaster	5 NAT (numerical answer type) questions
`formula_card`	Atlas Writer	Single-screen flash card for one formula
`concept_map`	Atlas Visualizer	Diagram of how concepts connect
`analogy`	Atlas Writer	Real-world comparison for abstract concept
`mnemonics`	Atlas Writer	Memory aids for formula recall

`short_video_script`	Atlas Writer	60–90 second explainer script

How content is delivered to students:

```
Atlas generates content atoms
    │
    └── CONTENT_READY → Sage
        → Sage verifies accuracy (samples 10–15% of atoms)
        → CONTENT_VERIFIED → Forge
            → Forge builds and deploys to CDN
            → EXAM_DEPLOYED → Oracle + Herald + Mentor
                → Oracle: start tracking engagement per atom
                → Herald: announce new content on social channels
                → Mentor: notify enrolled students of new content
```

How content adapts over time:

Signal	From	To	What changes
`CONTENT_GAP`	Sage	Atlas	New explanation generated for a topic a student couldn't understand
`STRUGGLE_PATTERN`	Sage	Atlas	Entire topic gets re-approached with a different format
`FORMAT_SUCCESS`	Sage	Atlas	Atlas generates more content in the format that worked
`ENGAGEMENT_GAP`	Mentor	Atlas	Topic that students skip gets a fresh, more engaging variant
`CONTENT_STALE`	Oracle	Atlas	Topic engagement dropped 30-day rolling avg; Atlas regenerates
`FUNNEL_INSIGHT`	Prism	Atlas	High-converting format gets prioritised in the next batch

How multiple students use the same content simultaneously:

1. Content is generated once and stored in the database
2. Each student's **session context** is personalised (persona, exam phase, weak topics, archetype)
3. Sage selects which atoms to use based on the student's current topic and mastery level
4. The same MCQ can be shown with different Sage guidance:
    - Grinder student: pushed harder immediately
    - Panicker student: given hint + encouragement first
    - Topper student: shown harder variant
5. Atlas generates topic **variants** — multiple explanations for the same concept at different difficulty levels and formats; Sage picks the best match for each student

---

11. Batch Jobs and Automation

When running on GCP or locally, you can trigger agent batch pipelines manually:

```
# Run all agent batch jobs in sequence
./scripts/batch-run.sh

# Run specific agent pipeline
./scripts/batch-run.sh --agent atlas      # Atlas content generation
./scripts/batch-run.sh --agent scout      # Scout market scan
./scripts/batch-run.sh --agent oracle     # Oracle analytics compute
./scripts/batch-run.sh --agent herald     # Herald campaign check
./scripts/batch-run.sh --agent mentor     # Mentor engagement nudges
```

```
./scripts/batch-run.sh --agent forge      # Forge health check

# With specific parameters
./scripts/batch-run.sh --agent atlas --exam GATE_EM --topics "Linear Algebra,Calculus"
./scripts/batch-run.sh --agent mentor --mode reengagement
./scripts/batch-run.sh --agent oracle --report weekly
```

Automated schedules (when running on GCP Cloud Scheduler):

Agent	Schedule	What it does
Atlas	Daily 7:30am IST	Generates next day's content batch
Scout	Monday 11:30am IST	Full market scan + competitor analysis
Oracle	Every 6 hours	KPI computation + churn risk scoring
Herald	Daily 1:30pm IST	Campaign performance check
Mentor	Daily 2:30pm IST	Sends nudges to at-risk students
Forge	Every 30 minutes	System health check + SSL monitoring

In-app manual triggers:

Page	Trigger	What happens
`/atlas-workbench`	"Generate Now"	Triggers Atlas for selected topic
`/batch-generate`	"Run Batch"	Triggers full Atlas batch for an exam
`/content-hub`	"Generate"	Single content atom via Atlas
`/social-intent`	"Scan Now"	Triggers Scout social intent pipeline
`/market-intel`	"Refresh"	Triggers Scout market scan
`/agents/:agentId`	"Trigger Heartbeat"	Forces an agent heartbeat

---

12. Skills System — VoltAgent Modules

Skills are toggleable AI capability modules loaded into specific agents at runtime. Manage them at `/agent-skills`.

Skill	Agents	What it does	Toggle
Guard Rails	Sage	Content safety, cheating detection, crisis detection	Yes
Thinking Tool	Sage, Atlas	6-phase structured reasoning for complex problems	Yes
Live Evals	Sage, Oracle	Real-time quality scoring of Sage responses (6 metrics)	Yes
Dynamic Prompts	All	Versioned, testable prompt templates; A/B variant selection	Yes
User Research	Scout, Oracle, Sage, Mentor	Student archetype profiling; injected into prompts	Yes

Media Content	Herald, Atlas	Platform-specific social media content generation	Yes
Voice Output	Sage, Mentor	Text-to-speech for Sage explanations	Yes
Social Intent Scout	Scout, Atlas, Herald, Forge	5-stage automated Q&A pipeline on social platforms	Yes

User Research Report (at `/agent-skills` → User Research Report section):

Click **Generate Report** to produce a per-student analysis across 10 dimensions:

Dimension	What you learn	Action generated
□ Archetype	Which of 5 archetypes the student matches	Sage adapts tone and pacing
□ Motivation	Why they're studying	Content framing matches motivation
⚠ Pain Points	What's blocking them	Atlas generates targeted remediation
□ Engagement	Session length, peak hours, trajectory	Sage and Mentor adjust delivery timing
□ Content Preferences	Format that works best	Atlas prioritises this format
□ Channel Behaviour	Where they engage most	All nudges sent on preferred channel
□ Knowledge Gaps	Weakest topics	Atlas queues content; Sage prioritises
□ Emotional State	Frustration or anxiety level	Sage switches to empathy mode
□ Learning Trajectory	Accelerating/steady/plateauing/declining	Difficulty adjusts accordingly
□ Churn Risk	Risk of student dropping off	Mentor triggers re-engagement

---

13. Troubleshooting

System won't start

```
# Check what's missing
./scripts/check-deps.sh

# Check Docker is running
docker ps

# Check port conflicts
lsof -i :3000   # backend
lsof -i :5432   # postgres
lsof -i :6379   # redis
```

No AI responses

1. Check LLM keys: `echo $GEMINI_API_KEY` — should not be empty
2. Test directly: `curl -H "Authorization: Bearer $GEMINI_API_KEY" https://generativelanguage.googleapis.com/v1beta/models`
3. Check fallback: if Gemini fails, does Anthropic key exist?
4. Check `/status` page — LLM provider health shown there

Content not generating

1. Go to `/agents` — check Atlas heartbeat status (should be `active`)

2. Check `/events` — has `EXAM_APPROVED` been emitted for your exam?
3. Check Atlas token budget: `/agents/atlas` → budget usage bar
4. Trigger manually: `/atlas-workbench` → Generate Now

Students not receiving nudges

1. Check Mentor status at `/agents/mentor`
2. Check channel config: WhatsApp requires `TWILIO_*` keys; Telegram requires `TELEGRAM_BOT_TOKEN`
3. Check `/events` — are `STUDENT_ENROLLED` signals visible?
4. Verify student's preferred channel in `/user-portal`

Database errors

```
# Check DB connection
psql $DATABASE_URL -c "SELECT 1;"

# Run migrations manually
npx supabase db push --project-ref your-ref
```

Build fails

```
cd frontend
npm install
npx tsc --noEmit    # TypeScript check — should exit 0
npm run build       # production build
```

Agent signals not flowing

1. Open `/events` in the CEO dashboard — live signal bus view
2. Check browser localStorage: `localStorage.getItem('edugenius_signals')` in DevTools console
3. If IndexedDB is full: DevTools → Application → Storage → Clear IndexedDB

Wolfram answers not working

- Confirm `VITE_WOLFRAM_APP_ID` is set in Netlify env vars
- Test: `curl "https://api.wolframalpha.com/v2/query?appid=YOUR_ID&input=eigenvalues&output=JSON"`
- Free tier: 2,000 queries/month

Reference — Document Index

Document	What it covers
00-overview.md	High-level system overview
01-quick-start.md	5-minute quick start
02-agent-architecture.md	All 8 agents + 45 sub-agents
03-llm-abstraction.md	LLM provider setup and fallback
04-event-system.md	Signal bus and event system
05-data-layer.md	Database schema and ORM
06-api-reference.md	REST/GraphQL API endpoints
07-workflows.md	Agent workflow definitions

08-testing-guide.md	Running tests
09-deployment.md	Full deployment reference
10-configuration.md	All environment variables
11-multi-agent-setup.md	OpenClaw multi-agent configuration
12-content-delivery.md	Content pipeline and CDN
12-go-live-checklist.md	Pre-launch checklist
13-deployment-modes.md	Deployment mode comparison
14-exam-configuration.md	Exam config structure
15-frontend-preview.md	Frontend preview and role switcher
16-website-portal-architecture.md	Public website and portal
17-master-design-documentation.md	1,500-line master design doc
18-agent-connection-map.md	Complete bidirectional signal map
19-audit-report.md	Bidirectional wiring audit findings
19-deployment-options.md	Deployment cost and comparison
20-content-system.md	Content system architecture
21-course-summary-outline.md	Course Summary Outline system
CEO-INTEGRATIONS-GUIDE.md	All integrations master guide
22-help-manual.md	**← You are here**